

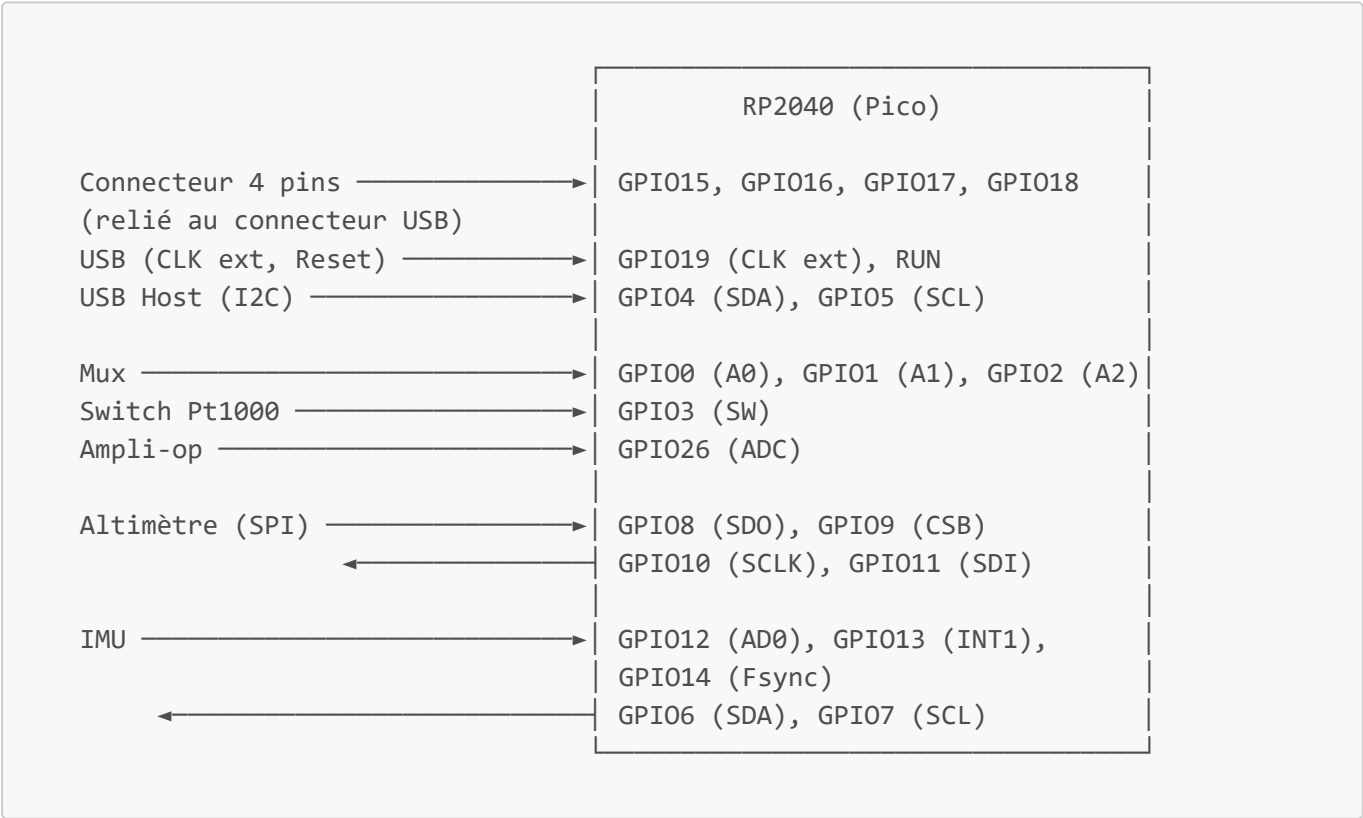
# SAFARI ADCS — Architecture V2

**Project:** Satellite Attitude & Formation Control — RP2040 Attitude Determination & Control System  
**Board:** Raspberry Pi Pico (RP2040)  
**Framework:** Arduino (PlatformIO, Mbed variant)  
**Revision:** V2 — 2026-05-28

## Diagrammes PlantUML

| Fichier                                        | Contenu                                        |
|------------------------------------------------|------------------------------------------------|
| <a href="#">diagrams/01_hardware.puml</a>      | Connexions matérielles RP2040 ↔ capteurs       |
| <a href="#">diagrams/02_classes.puml</a>       | Hiérarchie de classes firmware                 |
| <a href="#">diagrams/03_sequence_init.puml</a> | Séquence d'initialisation <code>setup()</code> |
| <a href="#">diagrams/04_sequence_loop.puml</a> | Cycle de mesure <code>loop()</code>            |

## 1. Système global



## 2. Carte des GPIOs

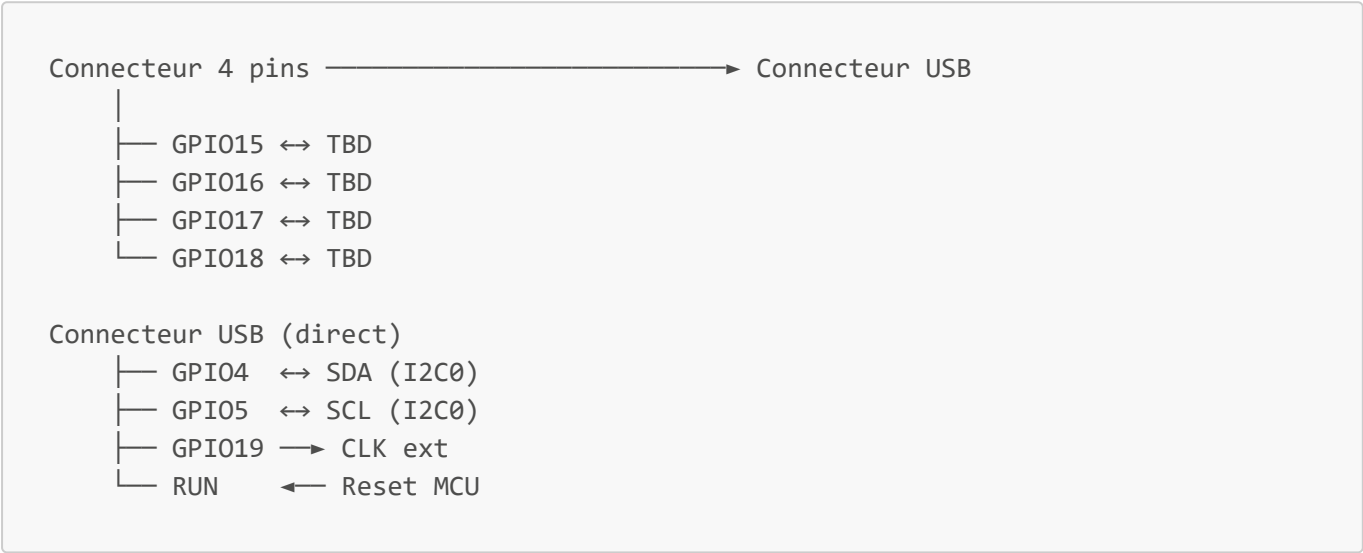
### 2.1 Tableau complet

| <b>GPIO</b> | <b>Dir MCU</b> | <b>Connecté à</b>       | <b>Signal</b>                              | <b>Sous-système</b> |
|-------------|----------------|-------------------------|--------------------------------------------|---------------------|
| GPIO0       | OUT            | Mux — broche A0         | Sélection Pt1000                           | Pt1000 / Mux        |
| GPIO1       | OUT            | Mux — broche A1         | Sélection Pt1000                           | Pt1000 / Mux        |
| GPIO2       | OUT            | Mux — broche A2         | Sélection Pt1000                           | Pt1000 / Mux        |
| GPIO3       | OUT            | Switch                  | ON/OFF Pt1000                              | Pt1000 / Mux        |
| GPIO4       | IN/OUT         | Connecteur USB (host)   | SDA USB                                    | Communication USB   |
| GPIO5       | IN/OUT         | Connecteur USB (host)   | SCL USB                                    | Communication USB   |
| GPIO6       | OUT            | IMU — broche SDA        | SDA (I2C)                                  | IMU                 |
| GPIO7       | OUT            | IMU — broche SCL        | SCL (I2C)                                  | IMU                 |
| GPIO8       | IN             | Altimètre — broche SDO  | SDO (Alt → MCU)                            | Altimètre           |
| GPIO9       | OUT            | Altimètre — broche CSB  | CSB / CS                                   | Altimètre           |
| GPIO10      | OUT            | Altimètre — broche SCLK | SCLK                                       | Altimètre           |
| GPIO11      | OUT            | Altimètre — broche SDI  | SDI (SPI)                                  | Altimètre           |
| GPIO12      | OUT            | IMU — broche AD0/SDO    | AD0 : sélection adresse I2C (LOW → 0x68)   | IMU                 |
| GPIO13      | IN             | IMU — broche INT1       | Interruption données prêtes (actif bas)    | IMU                 |
| GPIO14      | IN             | IMU — broche Fsync      | Frame Sync (latch accéléro sur front ext.) | IMU                 |
| GPIO15      | IN/OUT         | Connecteur 4 pins → USB | TBD                                        | Communication USB   |
| GPIO16      | OUT            | Connecteur 4 pins → USB | TBD                                        | Communication USB   |
| GPIO17      | IN             | Connecteur 4 pins → USB | TBD                                        | Communication USB   |
| GPIO18      | IN/OUT         | Connecteur 4 pins → USB | TBD                                        | Communication USB   |
| GPIO19      | IN/OUT         | Connecteur USB          | CLK ext USB                                | Communication USB   |

| GPIO   | Dir MCU  | Connecté à      | Signal        | Sous-système      |
|--------|----------|-----------------|---------------|-------------------|
| GPIO26 | IN (ADC) | Sortie ampli-op | MES Pt1000    | Pt1000 / Mux      |
| RUN    | OUT MCU  | Connecteur USB  | Reset externe | Communication USB |

### 3. Sous-systèmes matériels

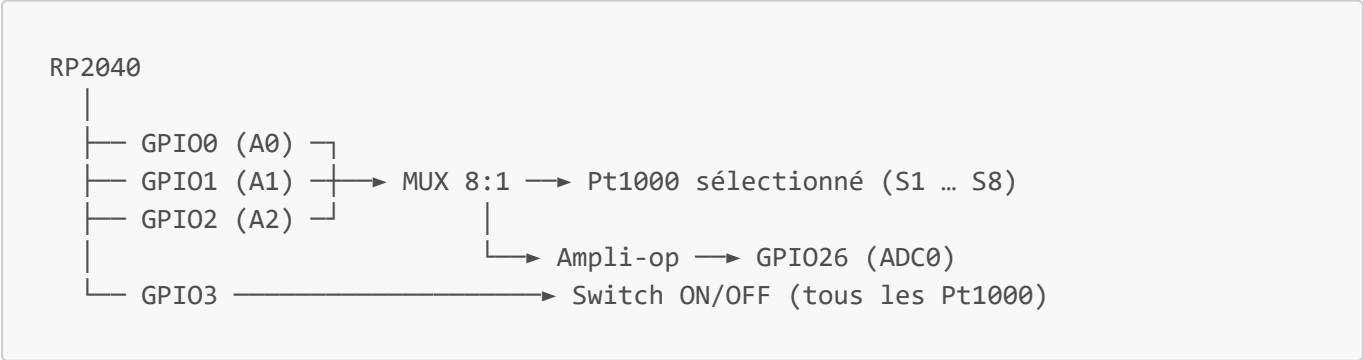
#### 3.1 Communication USB



- **GPIO15/18** : signaux non encore définis sur le connecteur 4 pins
- **GPIO16/17** : signaux non encore définis sur le connecteur 4 pins
- **GPIO4/5** : I2C0 — bus accessible depuis le connecteur USB
- **GPIO19** : horloge externe USB
- **RUN** : reset hardware du RP2040 depuis le connecteur USB

#### 3.2 Pt1000 — Multiplexeur analogique

Architecture : MCU → MUX (8 canaux) → Pt1000 (1 parmi 8) → Ampli-op → MCU (ADC)



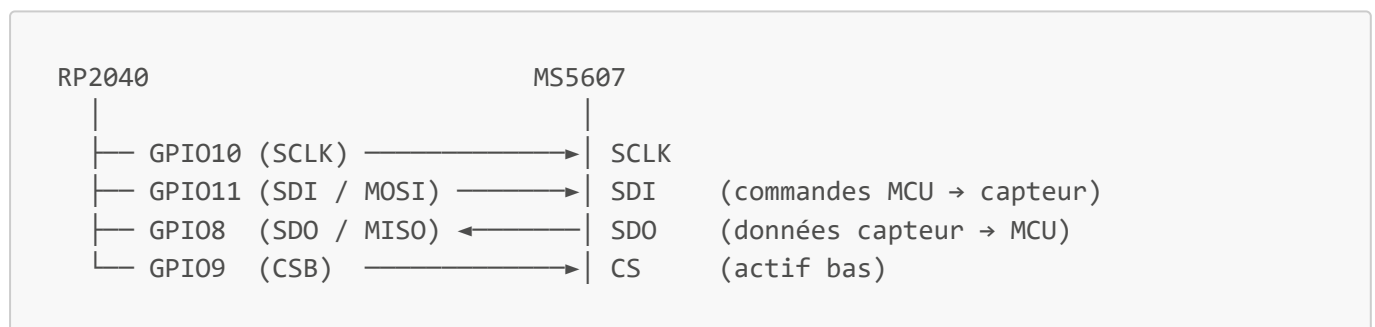
Logique de sélection (A2:A1:A0) :

| A2 | A1 | A0 | Pt1000 sélectionné |
|----|----|----|--------------------|
| 0  | 0  | 0  | S1                 |
| 0  | 0  | 1  | S2                 |
| 0  | 1  | 0  | S3                 |
| 0  | 1  | 1  | S4                 |
| 1  | 0  | 0  | S5                 |
| 1  | 0  | 1  | S6                 |
| 1  | 1  | 0  | S7                 |
| 1  | 1  | 1  | S8                 |

- GPIO3 met tous les Pt1000 hors tension (LOW = OFF) — permet d'économiser l'énergie entre mesures
- GPIO26 (ADC0) lit la tension de sortie de l'ampli-op proportionnelle à la résistance du Pt1000 actif
- La conversion résistance → température se fait en firmware (courbe de calibration Pt1000 standard)

### 3.3 Altimètre — MS5607-02BA03-50 (TE Connectivity)

Communication exclusivement en **SPI mode 0** (CPOL=0, CPHA=0) à **1 MHz** :



**Commandes firmware (OSR = 4096, précision maximale) :**

| Commande                 | Octet     | Délai | Usage                        |
|--------------------------|-----------|-------|------------------------------|
| Reset                    | 0x1E      | 3 ms  | Initialisation obligatoire   |
| Convert D1 (P)           | 0x48      | 10 ms | Lance conversion pression    |
| Convert D2 (T)           | 0x58      | 10 ms | Lance conversion température |
| ADC Read                 | 0x00      | —     | Lit résultat 24 bits         |
| PROM Read C <sub>i</sub> | 0xA0 + 2i | —     | Lit coefficient calibration  |

#### Séquence d'initialisation :

1. CS haut → Reset → attendre 3 ms
2. Lire PROM : 8 mots de 16 bits (C0...C7), vérifier C1...C6 ≠ 0

#### Cycle de mesure :

1. Conv D1 → attendre 10 ms → ADC Read → D1 (pression brute)
2. Conv D2 → attendre 10 ms → ADC Read → D2 (température brute)
3. Compensation 2nd ordre (algo datasheet MS5607 rev. 1.2) → P (hPa), T (°C)
4. Altitude :  $44330 \times (1 - (P / P_0)^{0.190295})$  m

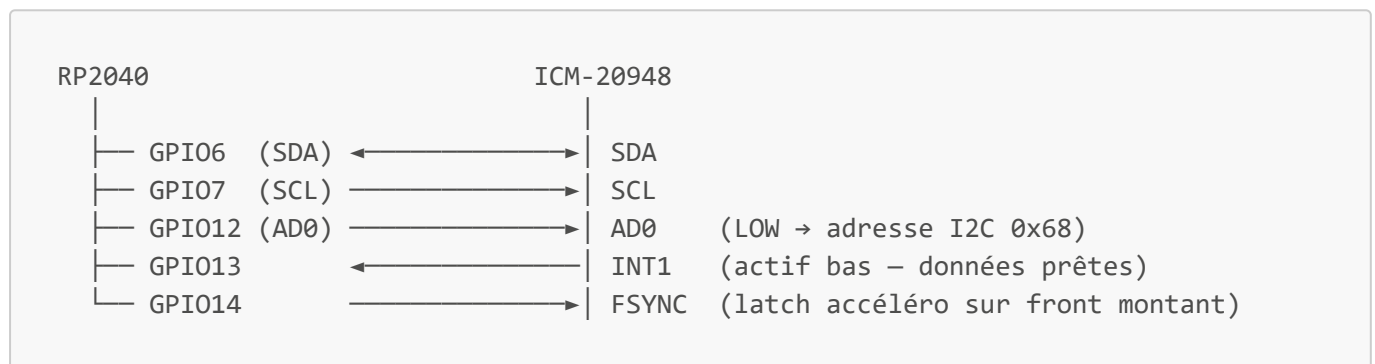
### Plages de mesure :

- Pression : 10 – 1200 hPa
- Température : -40°C à +85°C

## 3.4 IMU — ICM-20948 (Adafruit #4554)

9 axes : accéléromètre + gyroscope (ICM-20948) + magnétomètre AK09916 intégré.

Bus **I2C1** à **400 kHz** (Wire1, GPIO6/7) :



### Détection firmware :

- Registre WHO\_AM\_I (0x00, bank 0) → réponse attendue **0xEA**

### Paramètres d'initialisation (Adafruit ICM20X) :

| Paramètre         | Valeur configurée | Plage possible                  |
|-------------------|-------------------|---------------------------------|
| Plage accéléro    | ±4 g              | ±2 / ±4 / ±8 / ±16 g            |
| Plage gyroscope   | ±500 °/s          | ±250 / ±500 / ±1000 / ±2000 °/s |
| Fréquence magnéto | 10 Hz (AK09916)   | 10 / 20 / 50 / 100 Hz           |

### Signaux de contrôle :

- **INT1** (GPIO13, actif bas) : déclenche une interruption quand un échantillon est prêt — permet la lecture sans polling
- **FSYNC** (GPIO14) : synchronise le timestamp accéléro sur un événement externe (GPS PPS, caméra...)
- **AD0** (GPIO12) : fixé LOW au démarrage → adresse fixe **0x68**, ne change pas en cours d'exécution

### Données produites :

- Accélération 3 axes (g)
- Vitesse angulaire 3 axes (°/s)
- Champ magnétique 3 axes (μT) — AK09916 via bus I2C secondaire interne
- Température interne (°C)

## Représentation d'attitude — quaternion unitaire $q = [w, x, y, z]$ :

Les données brutes des 3 capteurs sont fusionnées pour produire un quaternion représentant la rotation du repère corps vers le repère de référence :

| Source            | Rôle dans la fusion                                                               |
|-------------------|-----------------------------------------------------------------------------------|
| Gyroscope (°/s)   | Intégration de la vitesse angulaire → propagation du quaternion à haute fréquence |
| Accéléromètre (g) | Correction du tangage et du roulis à basse fréquence (référence gravité)          |
| Magnétomètre (μT) | Correction du lacet (référence champ magnétique terrestre)                        |

Le quaternion évite le blocage de cardan (*gimbal lock*) inhérent aux angles d'Euler. L'algorithme de fusion (filtre complémentaire, Madgwick ou Mahony) sera sélectionné après caractérisation des capteurs.

## 4. Architecture firmware — Modularité

### 4.1 Principe

Chaque capteur est un module **totalelement indépendant**. L'absence d'un capteur sur le PCB ne doit jamais provoquer de crash ou de blocage. Le firmware détecte la présence de chaque capteur à l'initialisation et désactive silencieusement les modules absents.

```
main.cpp
├── sensor_manager      ← orchestre les modules, aucune logique capteur ici
│   ├── imu_module      ← optionnel
│   ├── altimeter_module ← optionnel
│   ├── pt1000_module   ← optionnel
│   └── usb_module       ← optionnel
└── data_bus            ← structure de données partagée (lecture seule depuis
    les modules)
```

### 4.2 Interface commune (*SensorModule*)

**V2 — rupture avec le code existant.** L'ancienne classe *Sensor* (*begin()* / *update()* / *getData()*) est remplacée par *SensorModule*. Toutes les implémentations V2 dérivent exclusivement de *SensorModule*.

Chaque module implémente la même interface minimaliste :

```
class SensorModule {
public:
    virtual bool    init()          = 0;  // false si capteur absent/défaillant
    virtual bool    read()          = 0;  // false si lecture échoue
    virtual bool    isReady() const = 0;
    virtual const char* name()      const = 0;
```

```
virtual ~SensorModule() {}
};
```

Règles :

- `init()` ne bloque **jamais** si le capteur ne répond pas (timeout  $\leq 50$  ms)
- `read()` ne bloque **jamais** (retourne `false` si donnée non disponible)
- Aucun module n'appelle un autre module directement

#### 4.2.1 Codes d'erreur

`init()` et `read()` retournent un `bool` pour la compatibilité avec `SensorManager`, mais chaque module doit exposer un code d'erreur détaillé via `lastError()` :

```
enum class SensorError : uint8_t {
    OK                = 0,
    NOT_FOUND,        // pas de réponse sur le bus (I2C NACK, SPI timeout)
    BAD_ID,            // WHO_AM_I inattendu ou CRC PROM invalide
    DATA_INVALID,     // valeur hors plage physique
    READ_FAIL          // lecture partielle ou corrompue
};
```

#### 4.2.2 Politique de récupération

| Événement                   | Réaction immédiate                                                   | Réaction après 3 échecs consécutifs              |
|-----------------------------|----------------------------------------------------------------------|--------------------------------------------------|
| <code>init()</code> → false | module marqué non-prêt, log Serial                                   | aucune — le module reste désactivé               |
| <code>read()</code> → false | donnée ignorée, <code>valid = false</code>                           | notification via <code>WatchdogComm</code> (USB) |
| Bus I2C bloqué              | reset du bus ( <code>Wire.end()</code> / <code>Wire.begin()</code> ) | module désactivé définitivement                  |

**Watchdog externe** : le module watchdog est un système séparé, codé indépendamment, qui communique avec l'ADCS **via USB**. `WatchdogComm` est l'interface côté ADCS qui gère cette liaison — réception des données du watchdog et émission de notifications d'état. Le protocole USB applicatif est défini du côté du module watchdog.

#### 4.3 Gestionnaire de capteurs (`SensorManager`)

```
class SensorManager {
    SensorModule* modules[MAX_MODULES];
    uint8_t count;
public:
    void registerModule(SensorModule* m);
    void initAll();    // init() sur chacun, log les absents
    void readAll();    // read() sur les présents seulement
};
```

- `initAll()` itère sur les modules enregistrés, marque `isReady = false` si `init()` échoue
- `readAll()` ignore silencieusement les modules non prêts
- `main.cpp` choisit quels modules enregistrer (via `#define ENABLE_IMU`, etc.)

## 4.4 Bus de données partagé

Structure en lecture seule entre modules (chaque module écrit uniquement sa propre section) :

```
struct AdcsData {
    struct {
        float ax, ay, az;           // m/s2
        float gx, gy, gz;           // °/s
        float mx, my, mz;           // μT
        bool valid;
    } imu;

    struct {
        float pressure;             // hPa
        float temperature;          // °C
        float altitude;             // m
        bool valid;
    } altimeter;

    struct {
        float temps[8];             // °C, index 0..7 = S1..S8
        bool valid[8];
        uint8_t active_count;
    } pt1000;

    struct {
        bool connected;
    } usb;
};

extern AdcsData g_adcs;
```

## 4.5 Configuration par `#define`

Dans `config.h`, chaque capteur est activable/désactivable à la compilation :

```
// Activer/désactiver les modules capteurs
#define ENABLE_IMU      1
#define ENABLE_ALTITUDE 1
#define ENABLE_PT1000   1
#define ENABLE_USB      1

// Nombre de Pt1000 connectés (1 à 8)
#define PT1000_COUNT    8
```



## 5. Flux d'exécution

```

setup()
├── SensorManager::initAll()
│   ├── IMU::init()           → détecte sur I2C 0x68/0x69, timeout 50 ms
│   ├── Altimeter::init()     → détecte sur SPI (CSB GPIO9), timeout 50 ms
│   ├── Pt1000::init()       → vérifie ADC GPIO26 lisible, teste GPIO0-3
│   └── USB::init()          → vérifie GPIO4/5 bus I2C
loop()
├── SensorManager::readAll()  ← exécuté à la fréquence souhaitée
│   ├── IMU::read()          → lit via I2C → g_adcs.imu
│   ├── Altimeter::read()    → lit via SPI → g_adcs.altimeter
│   ├── Pt1000::read()       → séquence mux A0-A2, lit ADC → g_adcs.pt1000
│   └── USB::read()          → sondage état bus → g_adcs.usb

```

## 6. Dépendances inter-modules

Aucune dépendance directe entre modules capteurs.

| Module    | Dépend de                                              |
|-----------|--------------------------------------------------------|
| imu       | GPIO6, GPIO7, GPIO12, GPIO13, GPIO14                   |
| altimeter | GPIO8 (SDO), GPIO9 (CSB), GPIO10 (SCLK), GPIO11 (SDI)  |
| pt1000    | GPIO0 (A0), GPIO1 (A1), GPIO2 (A2), GPIO3 (SW), GPIO26 |
| usb       | GPIO4, GPIO5, GPIO15–18, GPIO19, RUN                   |
| main      | SensorManager, AdcsData (g_adcs)                       |

Si un GPIO est absent ou en court-circuit, seul le module correspondant est marqué non prêt. Les autres continuent de fonctionner normalement.

## 7. Résumé des bus de communication

| Bus  | GPIOs          | Périphérique                  | Vitesse / protocole           |
|------|----------------|-------------------------------|-------------------------------|
| I2C0 | GPIO4, GPIO5   | Connecteur USB (SDA/SCL)      | 400 kHz                       |
| I2C1 | GPIO6, GPIO7   | IMU ICM-20948 (0x68)          | 400 kHz                       |
| SPI  | GPIO8–11       | Altimètre MS5607 (CS=GPIO9)   | 1 MHz, mode 0 (CPOL=0,CPHA=0) |
| GPIO | GPIO16, GPIO17 | Connecteur 4 pins → USB (TBD) | —                             |
| ADC  | GPIO26         | Pt1000 via Mux + Ampli-op     | 12 bits, Vref 3.3 V           |
| GPIO | GPIO0–3        | Mux A0/A1/A2 + Switch Pt1000  | —                             |
| GPIO | GPIO12         | IMU AD0 (adresse I2C fixe)    | —                             |

| Bus  | GPIOs          | Périphérique                           | Vitesse / protocole |
|------|----------------|----------------------------------------|---------------------|
| GPIO | GPIO13         | IMU INT1 (interruption données prêtes) | actif bas           |
| GPIO | GPIO14         | IMU FSYNC (latch accéléro)             | front montant       |
| GPIO | GPIO15, GPIO18 | Connecteur 4 pins → USB (TBD)          | —                   |

## 8. Librairies et dépendances

### 8.1 Librairies externes — PlatformIO (`lib_deps`)

| Librairie               | Version | Rôle dans le firmware                                |
|-------------------------|---------|------------------------------------------------------|
| Adafruit ICM20X         | ^2.0.5  | Driver ICM-20948 : lecture accéléro / gyro / magnéto |
| Adafruit BusIO          | ^1.14.1 | Abstraction I2C et SPI — dépendance d'ICM20X         |
| Adafruit Unified Sensor | ^1.1.9  | Interface capteur générique — dépendance d'ICM20X    |

Adafruit BusIO et Adafruit Unified Sensor sont des dépendances transitives : elles sont installées automatiquement par PlatformIO avec Adafruit ICM20X.

### 8.2 Framework et bibliothèques SDK

| Composant | Fourni par                                                     | Usage                                                                                                                                             |
|-----------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Arduino.h | Platform <code>raspberrypi</code> ,<br>board <code>pico</code> | Core Arduino : <code>millis()</code> , <code>delay()</code> , <code>pinMode()</code> ,<br><code>digitalWrite()</code> , <code>analogRead()</code> |
| Wire.h    | Arduino-Mbed / arduino-<br>pico                                | I2C0 (GPIO4/5) et I2C1 (GPIO6/7)                                                                                                                  |
| SPI.h     | Arduino-Mbed / arduino-<br>pico                                | SPI vers MS5607 (GPIO8–11)                                                                                                                        |
| math.h    | Toolchain ARM (libc)                                           | <code>powf()</code> pour le calcul d'altitude barométrique                                                                                        |
| stdint.h  | Toolchain ARM (libc)                                           | Types <code>uint8_t</code> , <code>uint16_t</code> , <code>uint32_t</code> , <code>int64_t</code>                                                 |

### 8.3 Modules internes

| Module  | Fichiers                                              | Dépend de                                                                                                          |
|---------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| config  | <code>include/config.h</code>                         | <code>stdint.h</code>                                                                                              |
| sensors | <code>include/sensors.h</code>                        | <code>stdint.h</code> — définit <code>SensorModule</code> (V2,<br>remplace l'ancienne classe <code>Sensor</code> ) |
| imu     | <code>include/imu.h</code> , <code>src/imu.cpp</code> | <code>sensors.h</code> , <code>config.h</code> , <code>Wire.h</code> , Adafruit<br>ICM20X                          |

| Module      | Fichiers                                      | Dépend de                                                                     |
|-------------|-----------------------------------------------|-------------------------------------------------------------------------------|
| altimeter   | include/altimeter.h,<br>src/altimeter.cpp     | sensors.h, config.h, SPI.h, math.h                                            |
| pt1000      | include/pt1000.h, src/pt1000.cpp              | sensors.h, config.h, Arduino.h                                                |
| watchdog    | include/watchdog.h,<br>src/watchdog.cpp       | stdint.h — interface USB vers le module<br>watchdog externe (codé séparément) |
| printStatus | include/printStatus.h,<br>src/printStatus.cpp | sensors.h, imu.h, altimeter.h, pt1000.h                                       |

## 8.4 Arbre de dépendances

```

main.cpp
├── Arduino.h
├── Wire.h
├── SPI.h
├── config.h
├── sensors.h
├── imu.h / imu.cpp
│   ├── sensors.h
│   ├── config.h
│   ├── Wire.h
│   └── Adafruit_ICM20948.h
│       ├── Adafruit ICM20X      ^2.0.5
│       ├── Adafruit BusIO      ^1.14.1 (transitive)
│       └── Adafruit Unified Sensor ^1.1.9 (transitive)
├── altimeter.h / altimeter.cpp
│   ├── sensors.h
│   ├── config.h
│   ├── SPI.h
│   └── math.h
├── pt1000.h / pt1000.cpp
│   ├── sensors.h
│   ├── config.h
│   └── Arduino.h
├── watchdog.h / watchdog.cpp
│   └── stdint.h
└── printStatus.h / printStatus.cpp
    ├── sensors.h
    ├── imu.h
    ├── altimeter.h
    └── pt1000.h

```

## 8.5 Notes sur les algorithmes embarqués (sans librairie externe)

| Algorithme                                  | Module                 | Norme / source                                                                           |
|---------------------------------------------|------------------------|------------------------------------------------------------------------------------------|
| Compensation pression/température 2nd ordre | <code>altimeter</code> | Datasheet MS5607 rev. 1.2                                                                |
| Équation maison tension $\rightarrow$ T     | <code>pt1000</code>    | Calibration propre au circuit (ampli-op + Mux) — remplace Callendar-Van Dusen            |
| Loi barométrique P $\rightarrow$ altitude   | <code>altimeter</code> | Formule hypsométrique standard                                                           |
| Fusion IMU $\rightarrow$ quaternion         | <code>imu</code>       | Algorithme à définir après caractérisation capteurs (Madgwick / Mahony / complémentaire) |
| Filtres numériques                          | tous modules           | À ajouter après tests sur capteurs réels (passe-bas, moyenne glissante...)               |